Объектно-ориентированное программирование

6 семестр
Лекция 2.

Содержание лекции:

- Объектно-реляционное сопоставление
- Работа с базой данных
- Entity Framework

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Работа с базой данных, Entity Framework

Ба́за да́нных — совокупность данных, хранимых в соответствии со схемой данных, манипулирование которыми выполняют в соответствии с правилами средств моделирования данных.

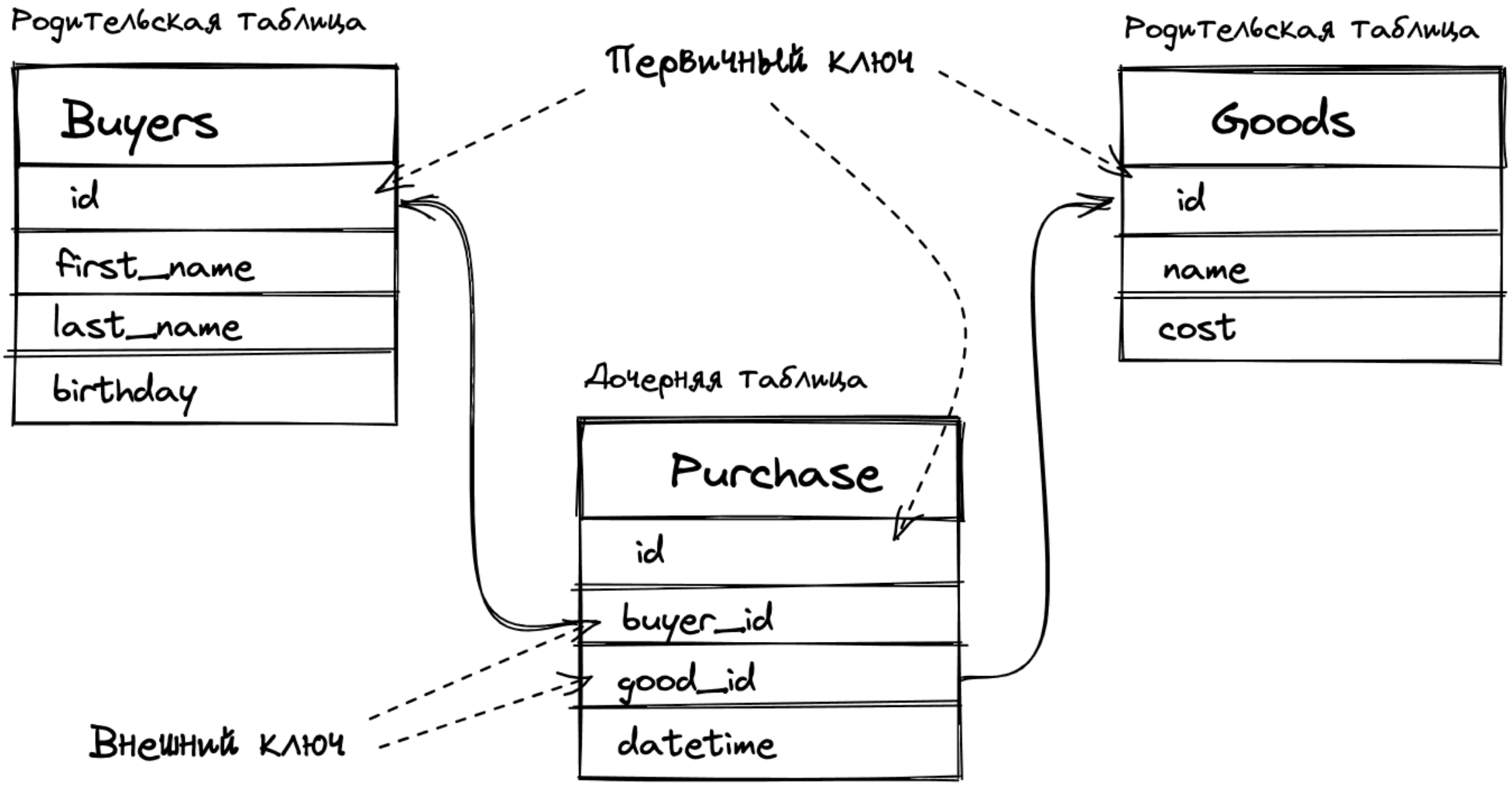
Систе́ма управле́ния ба́зами да́нных, сокр. СУБД — совокупность программных и лингвистических средств общего или специального назначения, обеспечивающих управление созданием и использованием баз данных. СУБД — комплекс программ, позволяющих создать базу данных и манипулировать данными.

Примерный вид базы данных

Сводная ведомость			
Дисциплина	Студент	Оценка	Дата сдачи
Информатика	1	Зачёт	24.10.2017
Математика	2	Отл.	16.10.2017
Математика	3	Зачёт	29.09.2017
Информатика	4	Хор.	02.11.2017

Студенты		
Фамилия	год рождения	Группа
Алфёрова	1995	103
Маслов	1994	100
Сафина	1996	105

Преподаватели	
Фамилия	Предмет
Иванов	Математика
Павлов	Информатика
Сафонова	Математика


Родительская таблица
Buyers
id
first_name
last_name
birthday
Первичный ключ
Родительская таблица
Goods
id
name
cost
Дочерняя таблица
Purchase
id
buyer_id
good_id
datetime
Внешний ключ

ADO.NET Entity Framework — объектно-ориентированная технология доступа к данным, является object-relational mapping (отображение объектов и связей) решением для .NET Framework от Microsoft. Предоставляет возможность взаимодействия с объектами как посредством LINQ в виде LINQ to Entities, так и с использованием Entity SQL.

Создадим WindowsForms (.Net 8) приложение. Для работы с БД будем использовать СУБД SQLite и EntityFramework.

Необходимо перейти в диспетчер пакетов Nuget и установить следующие пакеты:

- Microsoft.EntityFrameworkCore,
- Microsoft.EntityFrameworkCore.Sqlite.

Обзор Установлено Обновления Консолидировать

Включить предвар

.NET **Microsoft.EntityFrameworkCore** автор: Microsoft

Entity Framework Core is a modern object-database mapper for .NET. It support
SQLite, Azure Cosmos DB, MySQL, PostgreSQL, and other databases through a p

.NET **Microsoft.EntityFrameworkCore.Sqlite** автор: Microsoft

SQLite database provider for Entity Framework Core.

Далее необходимо создать классы, которые будут описывать модель БД. По сути классы здесь эквивалент записям в таблицах БД. Название класса – название таблицы; свойство класса – атрибут таблицы, экземпляр класса – 1 запись в таблице. Создадим покупателя:

```
public class Customer
{
        public int Id { get; set; }
        public string? Name { get; set; }
        public int Age { get; set; }
}
```

Чтобы задать своё имя таблица используется атрибут [Table(“Имя”)].

Чтобы явным образом указать первичный ключ нужно использовать атрибут [Key].

Чтобы задать минимальную или максимальную длину строки используются атрибуты [MinLength(N)] и [MaxLength(N)].

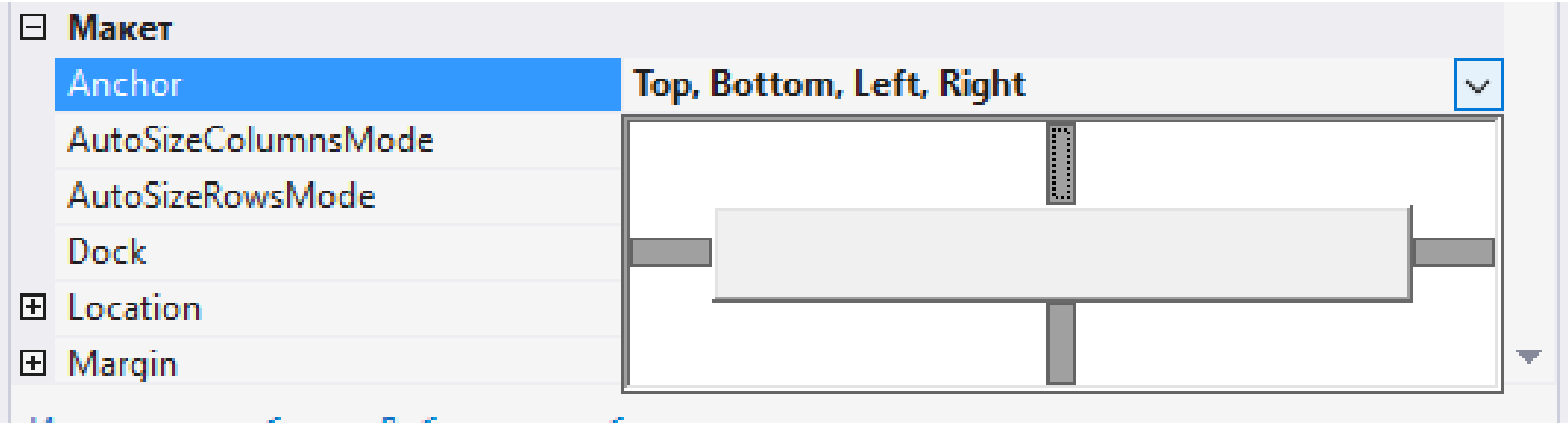
```
[Table(“Покупатель”)]
public class Customer
{
        [Key] public int Id { get; set; }
        [MaxLength(200)] public string? Name { get; set; }
        public int Age { get; set; }
}
```

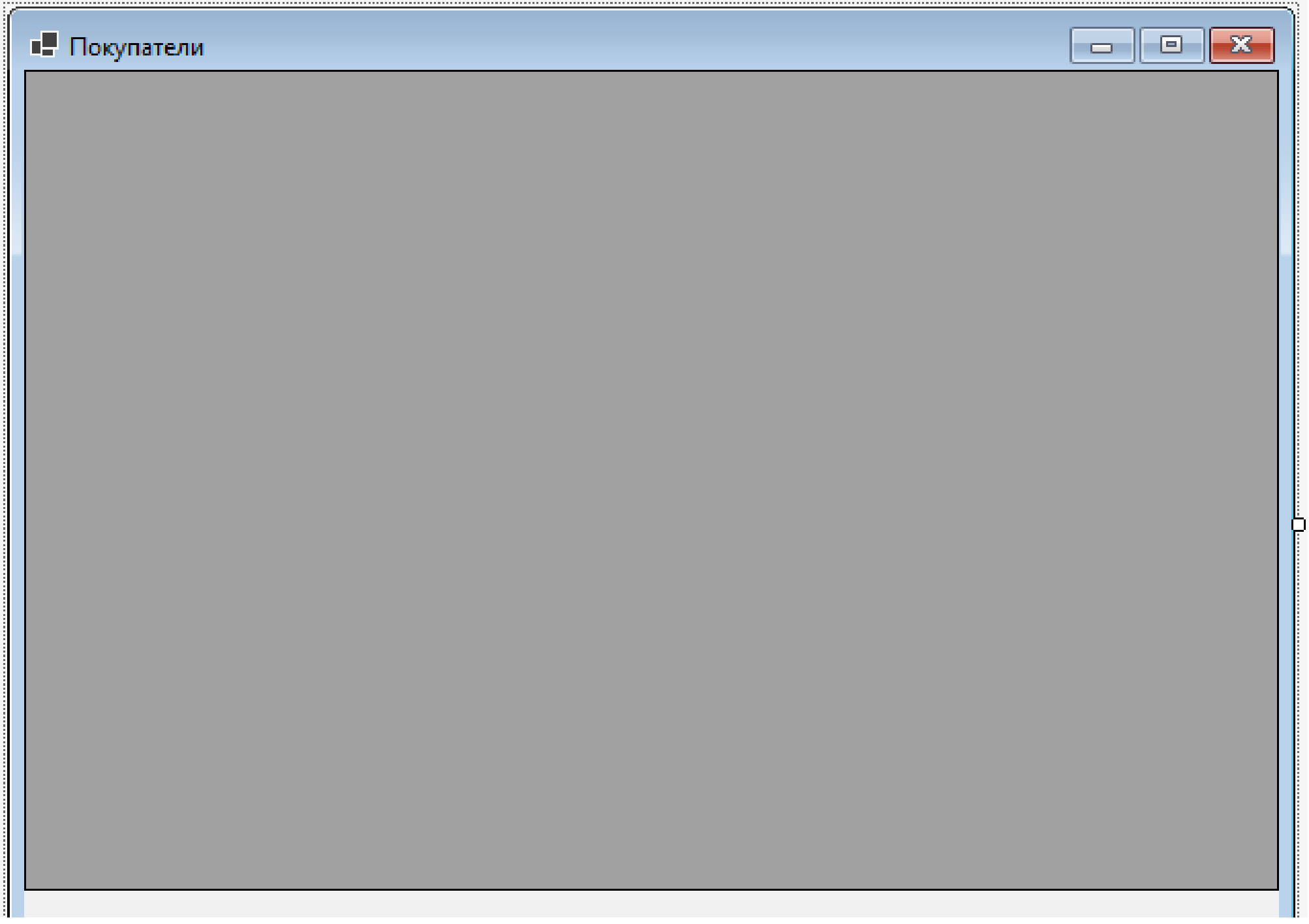
Взаимодействие с базой данных в **Entity Framework Core** происходит посредством специального класса - **контекста данных**. Поэтому добавим в наш проект новый класс, который назовем ApplicationContext и который будет иметь следующий код:

```
public class ApplicationContext : DbContext
{
//Здесь пишем все наборы данных (таблицы), что должны быть в БД
        public DbSet<Customer> Customers => Set<Customer>();
        public ApplicationContext() => Database.EnsureCreated();
        protected override void OnConfiguring(
                        DbContextOptionsBuilder optionsBuilder)
        {
          optionsBuilder.UseSqlite("Data Source=MyDataBase.db");
        }
}
```

стоит отметить, что по умолчанию у нас нет базы данных. Поэтому в конструкторе класса контекста определен вызов метода **Database.EnsureCreated()**, который при создании контекста автоматически проверит наличие базы данных и, если она отсуствует, создаст ее.

Добавим на форму DataGridView для отображения содержимого таблицы Customers. Установим свойство Anchor следующим образом:




Покупатели

Заполним БД некоторыми данными:

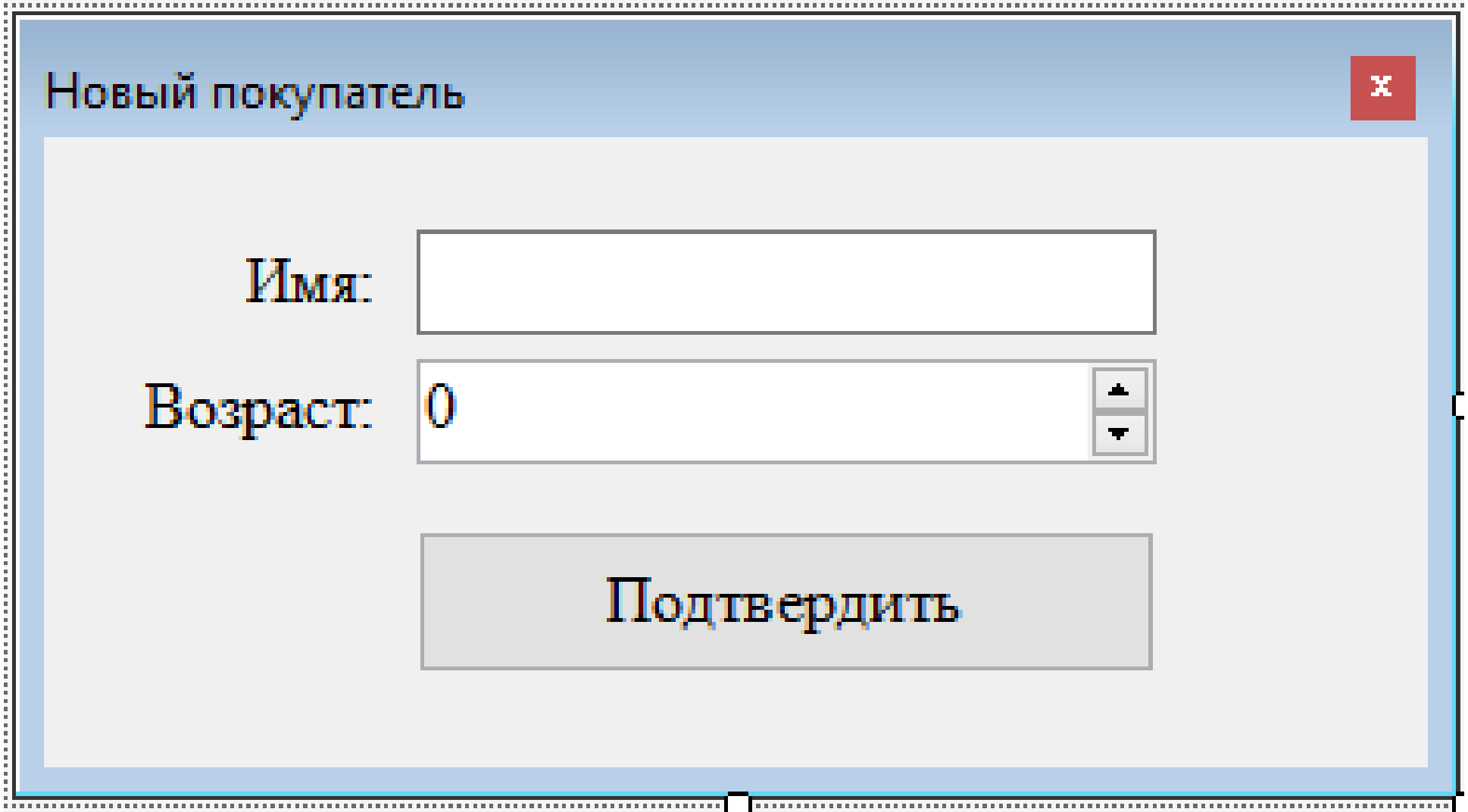
```
public Form1()
{
    InitializeComponent();

    using ApplicationContext db = new ApplicationContext();
    Customer tom = new() { Name = "Tom", Age = 33 };
    Customer alice = new() { Name = "Alice", Age = 26 };

    // добавляем их в бд
    db.Customers.Add(tom);
    db.Customers.Add(alice);
    db.SaveChanges();
    // получаем объекты из бд
    var customers = db.Customers.ToList();
    //Отображаем в таблице на форме
    dataGridView1.DataSource = customers;
}
```

Запустим программу:

Покупатели

	Id	Name	
▶	1	Tom	33
	2	Alice	26

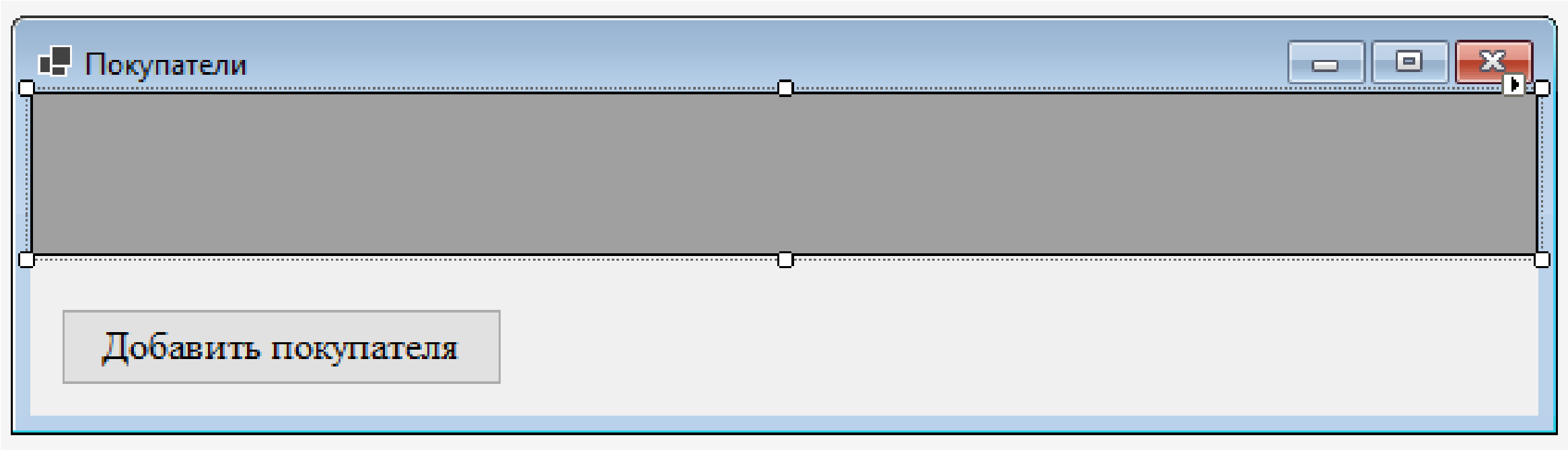
Сделаем добавление покупателя с помощью ввода данных с формы. Создадим форму для ввода данных покупателя. Свойства формы: FormBorderStyle = FixedToolWindows.



Содержимое формы для создания пользователя:

```
public partial class CustomerForm : Form
{
    public Customer? Customer { get; private set; }
    public CustomerForm()
    {
        InitializeComponent();
    }
    private void buttonAccept_Click(object sender,EventArgs e)
    {
        Customer = new Customer {
            Name = textBoxName.Text,
            Age = (int)numericUpDownAge.Value
        };
        DialogResult = DialogResult.OK;
        Close();
    }
}
```

Добавим на главную форму кнопку вызова формы добавления покупателя и поставим свойство Anchor: Bottom, Left.



Опишем метод, вызываемый при нажатии на кнопку:

```
private void buttonAddCustomer_Click(object sender, EventArgs e)
        {
            var newCustomerForm = new CustomerForm();

            if (newCustomerForm.ShowDialog() != DialogResult.OK ||
                newCustomerForm.Customer is null)
                return;

            var newCustomer = newCustomerForm.Customer;

            using ApplicationContext db = new();
            db.Customers.Add(newCustomer);
            db.SaveChanges();

            dataGridView1.DataSource = db.Customers.ToList();
        }
```

Покупатели

	Id	Name	Age
▶	1	Tom	33
	2	Alice	26

Новый покупатель

Имя: Иван

Возраст: 55

Подтвердить

Покупатели

	Id	Name	Age
▶	1	Tom	33
	2	Alice	26
	6	Иван	55

Добавим возможность удаления покупателей. Найдем событие **PreviewKeyDown** у **DataGridView** и подпишем метод удаления записей на это событие:

```
private void dataGridView1_PreviewKeyDown(
   object sender, PreviewKeyDownEventArgs e)
{

}
```

```
if (e.KeyCode == Keys.Delete)
{
  var customersToRemove = new List<Customer>();

  for (var row = 0;row<dataGridView1.SelectedRows.Count; row++)
  {
    var customer =
        (Customer)dataGridView1
                                .SelectedRows[row]
                                .DataBoundItem;

    customersToRemove.Add(customer);
  }

  RemoveCustomers(customersToRemove);
}
```

Метод удаления:

```
private void RemoveCustomers(IList<Customer> customers)
        {
            using ApplicationContext db = new();

            foreach (var customer in customers)
            {
                db.Customers.Remove(customer);
            }

            db.SaveChanges();
            dataGridView1.DataSource = db.Customers.ToList();
        }
```

Удаление происходит при нажатии клавиши Delete, выделяем покупателей и нажимаем Delete:

	Id	Name	Age
	1	Tom	33
	7	Петр	Tom
	8	Иван	55
▶	9	Вадим	33

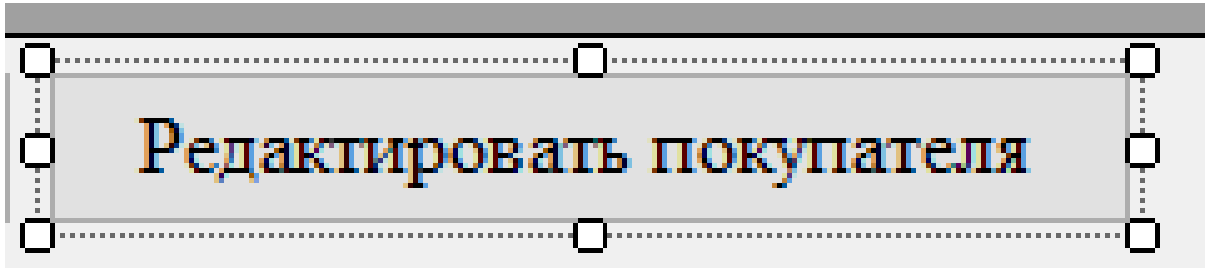
	Id	Name	Age
▶	1	Tom	33
	7	Петр	13

Добавим возможность изменения данных покупателя. Для этого модифицируем наш класс добавления покупателя, добавив конструктор:

```
public Customer Customer { get; private set; }=new Customer();
public CustomerForm()
{
   InitializeComponent();
}
public CustomerForm(Customer customer)
{
   InitializeComponent();

   Customer = customer;

   textBoxName.Text = customer.Name;
   numericUpDownAge.Value = customer.Age;
}
```

```
private void buttonAccept_Click(object sender, EventArgs e)
{
    Customer.Name = textBoxName.Text;
    Customer.Age = (int)numericUpDownAge.Value;


    DialogResult = DialogResult.OK;
    Close();
}
```

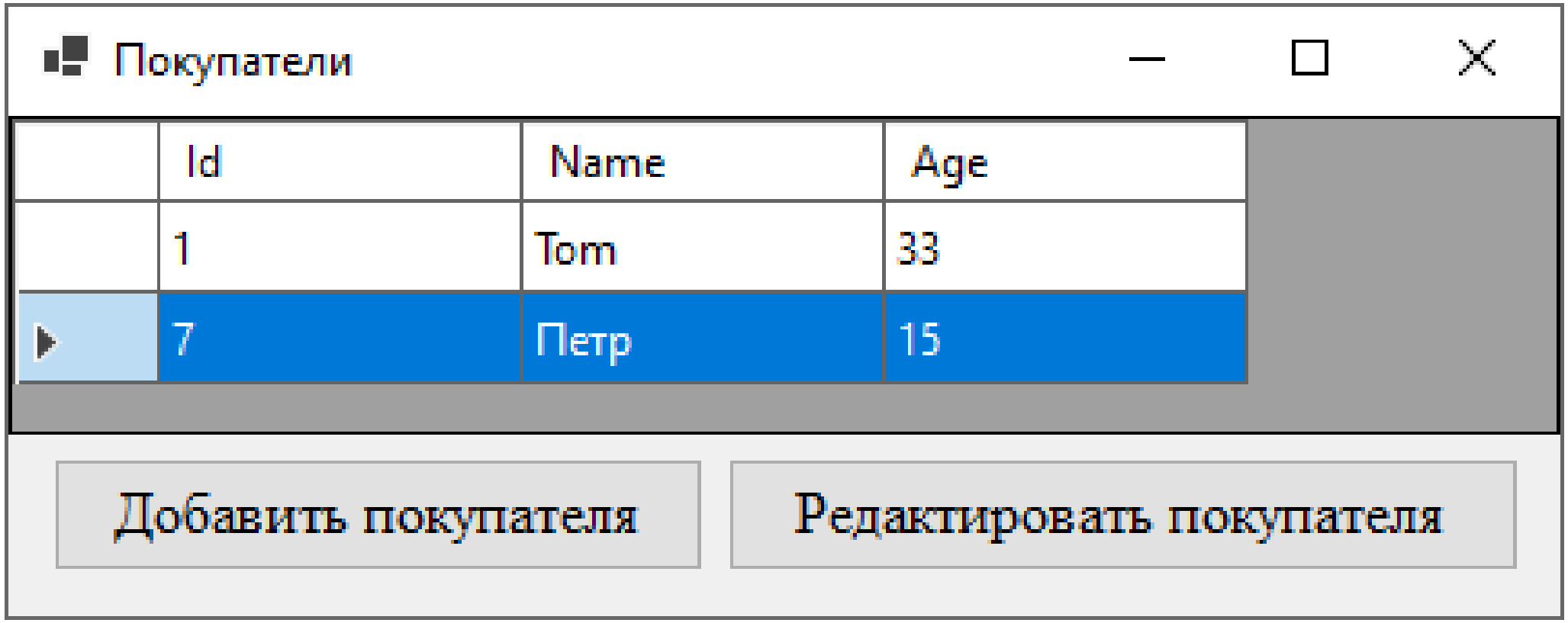
Добавим кнопку, при нажатии на которую будем редактировать выделенную запись таблицы:




```
private void buttonEditCustomer_Click(object sender, EventArgs e)
        {
            if (dataGridView1.SelectedRows.Count == 0)
                return;

            var customerId =
       ((Customer)dataGridView1.SelectedRows[0].DataBoundItem).Id;

            using ApplicationContext db = new();

            var customer = db.Customers
                  .SingleOrDefault(x => x.Id == customerId);
```

```
        if (customer == null)
            return;

        var customerForm = new CustomerForm(customer);

        if (customerForm.ShowDialog() != DialogResult.OK ||
            customerForm.Customer is null)
            return;

        db.SaveChanges();

        dataGridView1.DataSource = db.Customers.ToList();
    }
```

Выделяем строку и нажимаем на кнопку «Редактировать покупателя»:



Покупатель

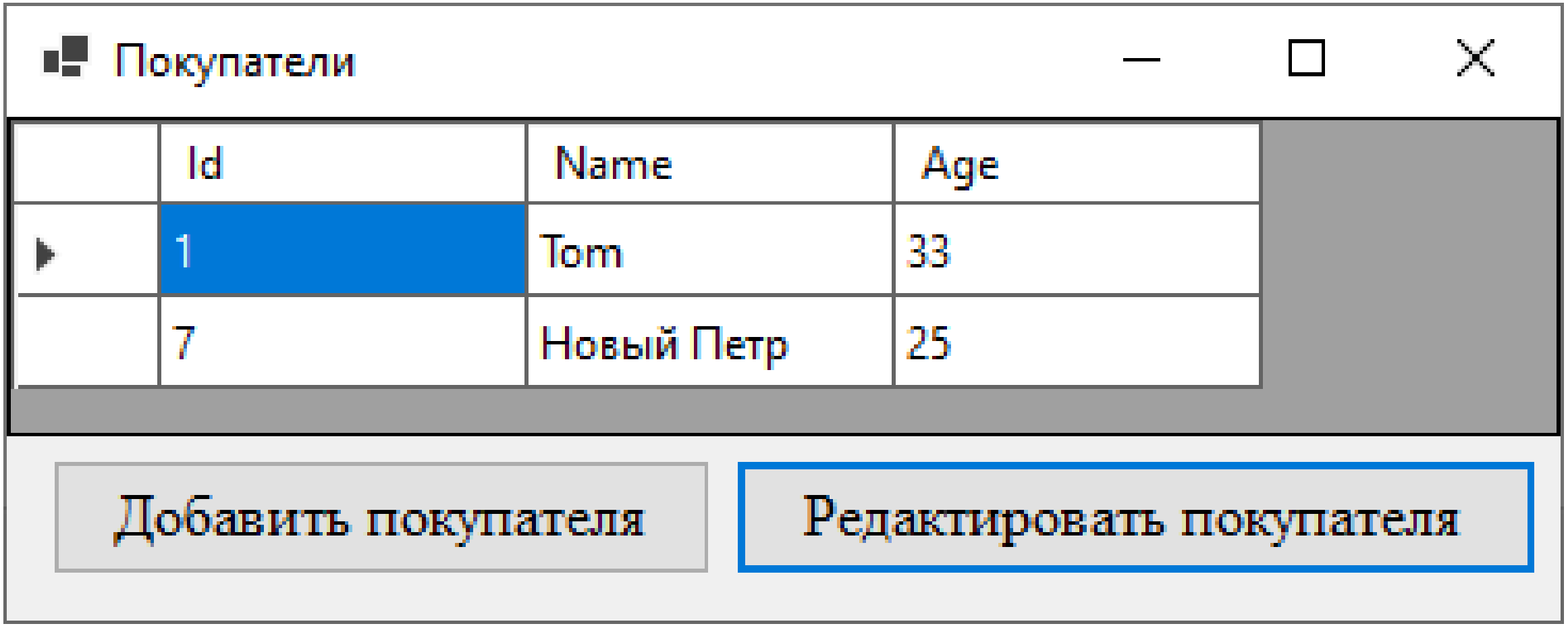
x

Имя: Петр

Возраст: 15

Подтвердить

Вводим данные и подтверждаем:



Все это время мы работали с одной таблицей, добавим ещё класс банковского аккаунта пользователя:

```
public class BankAccount
    {
        public int Id { get; set; }
        public int CustomerId { get; set; } //Внешний ключ
        public double Money { get; set; }
        public Customer? Customer { get; set; } //Навигационное
свойство
    }
```

Внешние ключи и навигационные свойства нужны для связи таблиц.

Добавим класс, описывающий товар:

```
public class Product
    {
        public int Id { get; set; }
        public string? Name { get; set; }
        public double Price { get; set; }
    }
```

Изменим класс покупателя, чтобы указать, что связь с банковским аккаунтом будет 1 к 1:

```
public class Customer
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public int Age { get; set; }

    //Добавили
    public BankAccount? BankAccount { get; set; }
}
```

Обновим файл нашей модели БД:

```
public class ApplicationContext : DbContext
{
      public DbSet<Customer> Customers {get; set;}
      public DbSet<Product> Products {get; set;}
      public DbSet<BankAccount> BankAccounts {get; set;}

      public ApplicationContext() => Database.EnsureCreated();

      protected override void OnConfiguring(
                              DbContextOptionsBuilder optionsBuilder)
      {
          optionsBuilder.UseSqlite("Data Source=MyDataBase.db");
      }
}
```

Посмотрим на примерах как можно работать с таблицами, заполним БД некоторыми данными:

```
using ApplicationContext db = new();
    db.Customers.AddRange(new List<Customer> {
        new Customer{Name = "Иван", Age = 25},
        new Customer{Name = "Петр", Age = 35},
        new Customer{Name = "Елена", Age = 23},
        new Customer{Name = "Анна", Age = 40},
    });

    db.Products.AddRange(new List<Product> {
        new Product{Name = "Стол", Price = 12000},
        new Product{Name = "Стул", Price = 2500},
        new Product{Name = "ВАЗ 2107", Price = 75000},
        new Product{Name = "Яблоки 1кг", Price = 120},
    });
    db.SaveChanges();
```

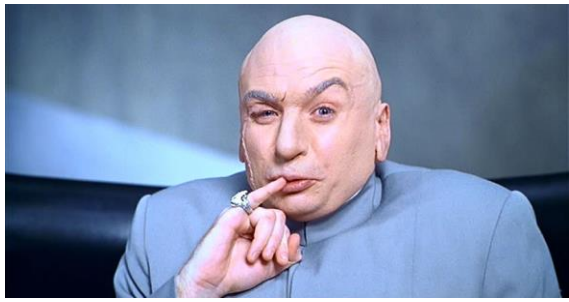
```
var customers = db.Customers.ToList();
db.BankAccounts.AddRange(new List<BankAccount>
{
  new BankAccount{Customer = customers[0], Money = 10000},
  new BankAccount{Customer = customers[1], Money = 250000},
  new BankAccount{Customer = customers[2], Money = 5000},
  new BankAccount{Customer = customers[3], Money = 1000000},
});

db.SaveChanges();
```

Примеры обращения к таблицам:

```
//Молодые покупатели
var youngCustomers = db.Customers
    .Where(c => c.Age <= 30)
    .ToList();

//Могут купить ВАЗ 2107
var canBuyVAZ = db.BankAccounts
    .Where(b => b.Money >= 75000)
    .Select(b => b.Customer)
    .ToList();
```

```
//Топ по кол-ву денег на счете
var moneyTop = db.BankAccounts
    .OrderByDescending(b => b.Money)
    .Select(b => new { b.Money, b.Customer });

//самый дорогой товар
var mostExpensive = db.Products
    .ToList()
    .MaxBy(p => p.Price);
```

Допустим мы хотим удалить из базы записи:



```
//Удаляем бедных
var customersToRemove = db.BankAccounts
      .Where(b => b.Money <= 1e4)
      .Select(b => b.Customer)
      .ToList();

  foreach (var customer in customersToRemove)
  {
      db.Customers.Remove(customer);
  }

  //Не забываем сохранять изменения
  db.SaveChanges();
```

Пример на изменение данных:

```
//Добавляем денег
db.BankAccounts
    .Where(b => b.Money <= 1e4)
    .ToList()
    .ForEach(b =>
    {
        b.Money *= 2;
    });

db.SaveChanges();
```